openSecureKey

Stand 27.10.2026. Dieser Plan ist am 27.10. vollständig neu geschrieben worden. Die gewachsene Fassung hatte 2071 Zeilen und sagte an zehn Stellen Widersprüchliches — der Akku war viermal verschieden beschrieben, der Massenspeicher dreimal, der Schlüsselexport dreimal. **Was hier steht, gilt.** Jede Entscheidung steht genau einmal; was offen ist, steht als offen.

Einordnung vorweg: Nichts von diesem Gerät ist je auf Hardware gelaufen. Rund 12 000 Zeilen C sind geschrieben und übersetzt, Schaltplan und Platine sind gezeichnet. Die Teile für den Teststand sind bestellt. **Das Projekt steht auf Stufe 2 von 4** — geschrieben / übersetzt / im Abbild / am Gerät gelaufen.

Was am 29.10.2026 selbst nachgemessen wurde, nicht aus Meldungen übernommen: sechs Prüfläufe der Stränge, jeder einzeln gestartet — `se050` 1066, `authz` 255, `ctap` 1425, `store` 88 631, `transport` 232, `applets` 1384 Fälle, **zusammen 92 993, kein Fehler**. *(Die 232 bei `transport` waren der Umfang von damals; heute sind es 1367 — der Satz ist seit dem 29.10. gewachsen, und dazwischen war er zeitweise rot, siehe unten.)* Die Fassung **ohne** RSA ist mitgemessen: `applets` 1222 Fälle, 0 Fehler — eine Option, die nur in einer Richtung geprüft ist, ist keine. Dazu `tools/check.py`: 14 Schritte ok, 308 Tests, 0 Fehler.

Am 30.10.2026 nachgemessen, nach vier weiteren Strängen: `check.py` 14 ok, 3 übersprungen, **330 Tests, 0 Fehler**; der UI-Prüfsatz **4074 Prüfungen, 0 Fehler** (gemessen auf `C:`, weil die WSL Netzlaufwerke nicht übersetzt); die Höhenrechnung `cad/tools/budget.py` **9 von 9 Kontrollpunkten, 4 davon mussten anschlagen**. **Und ein Befund, der die Zahlen oben relativiert:** die C-Prüfsätze `ctap` und `transport` waren vom 29.10.2026, 17:38 Uhr bis zum 30.10.2026 **rot** — niemandem aufgefallen, auch mir nicht. Strang E hat `transport/Makefile` um 17:19 zuletzt angefasst, Strang B hat um 17:38 drei Dateien zum CTAP-Kern hinzugefügt (`osk_sigcount.c`, `osk_ctap_cm.c`, `osk_ctap_config.c`) und sie nur in **seinem** Makefile nachgetragen. Dazu waren beide AAGUID-Konstanten unbedingt definiert, so dass in **jeder** Baurichtung eine davon unbenutzt blieb und `-Werror=unused-const-variable` abbrach — **dieser Code konnte in keiner Richtung übersetzen.** Behoben am 30.10.2026, danach gemessen: **ctap 1425 Fälle, transport 1367, je 0 Fehler, und beides in BEIDEN Richtungen der RSA-Option** (`make RSA=0` gibt es jetzt auch hier, wie bei `applets`). **Zwei Lehren, die teurer waren als die Behebung: Ein Strang, der nur seinen eigenen Prüfsatz messen kann, bricht den des Nachbarn unbemerkt** — und ich habe die sechs Prüfläufe *vor* dem Zusammenführen gemessen, nicht danach. **Nachgemessen wird am Ergebnis, nicht an den Teilen.**

Drei Schritte sind übersprungen und damit NICHT geprüft: `firmware/build` (kein nRF Connect SDK), `firmware/tests` und `firmware/format`. Die C-Prüfläufe der Stränge laufen deshalb ausdrücklich nicht über `check.py`, sondern über die Skripte in `apps/firmware/test/*/` — „check.py ist grün" sagt über die Firmware nichts.

---

1. Was das Gerät ist

Ein USB-Sicherheitsschlüssel, der zusammenbringt, was es sonst nur einzeln gibt: **FIDO2/Passkeys**, **Smartcard** mit eigenen Zertifikaten, **Einmalcodes** und einen **echten Passwortspeicher**. Dazu eine **Krypto-Geldbörse**, eine **Dongle-Funktion** für eigene Software und einen **verschlüsselten Datenträger**, den man weitergeben kann.

Alles selbst: Schaltplan, Platine, Firmware, Verwaltungsprogramm. Fremdcode wird nicht eingebunden; Hersteller-SDKs bleiben Werkzeug.

Was kein Mitbewerber hat: Passwortspeicher (768 Einträge je Profil, 3072 im Gerät — OnlyKey schafft 24, Nitrokey Pro 16, Yubico hat keinen), **Anzeige am Gerät** (Grundlage dafür, dem Rechner nicht glauben zu müssen), Geldbörse mit Betragsanzeige, Dongle-Betrieb, und ein Datenträger zum Weitergeben.

---

2. Hardware

Teil	Wahl	Anmerkung
Rechenkern	nRF52840 als Funkmodul **Raytac MDBT50Q-1MV2**	vorzertifiziert — sonst müsste die Platine selbst als Funkgerät geprüft werden

Teil	Wahl	Anmerkung
Sicherheitschip	**SE051C2** — gezeichnet, bestellt, und es bleibt dabei	Prüfbar nach Lieferung an der Kennung: **A8FA** oder **A564** = richtig, **A920** oder **A565** = falsche Variante, dann fehlen RSA und alle Edwards-Kurven
Flash	W25Q512JVEIQ, 64 MB (QSPI)	Tresor, Geräteprotokoll, Programmlaufwerk
Datenträger	**microSD-Schacht**, Push-Push	tauschbar — nur so sind die Betriebsarten 2 und 3 möglich; kostet keine Länge
Anzeige	Crystalfontz CFAL4864A-071BW, 0,71"	kleinster belegter; der nächstgrößere ist breiter als die ganze Platine
USB	GCT **USB4155-03-C** (LCSC C5183853)	Randstecker, für 0,8 mm belegt, Lötbildtiefe **3,303 mm** (gemessen, nicht 2,50 — die Schirmstifte reichen tiefer)
Fingerabdruck	FPC2534, **Bestückungsstufe 2**	Stiftreihe J5 liegt auf der Platine
Energie	**kein Akku**	siehe unten
Platine	**gezeichnet 27 × 70,0 mm**; gerechnete Schranke 59,55 mm	0,8 mm, 4 Lagen, beidseitig. **Die beiden Längen sind nicht dasselbe** — siehe offener Punkt 17

Kein Akku

Entschieden am 25.10.2026. Keine Zelle, kein Laderegler, keine Schutzschaltung; 14 Bauteile sind aus Schaltplan und Platine entfernt.

- **Die Uhr** wird beim Einstecken vom Rechner gestellt. Einmalcodes entstehen im Moment des Benutzens — und da steckt das Gerät. **Das war der Befund, der den Akku überflüssig machte.**
- **NFC und Bluetooth** gibt es nur am Rechner; der Rechenkern kann sich nicht aus dem NFC-Feld versorgen. Bewusst in Kauf genommen.
- **Gewonnen:** 10,45 mm Länge, kein Verbrauchsteil (ein LiPo im Anhänger hält wenige Jahre), keine Antennendämpfung, kein Laufzeitrisiko.
- **Der Manipulationsschutz braucht ihn nicht.** Epoxid-Verguss ersetzt die Reaktionsschaltung — belegt am iStorage datAshur PRO, einem Stick mit FIPS 140-3 **Overall Level 3 und ohne jede Tamper-Elektronik**.

> **Er käme nur unter drei Bedingungen zurück**, und alle drei werden eingehalten: ein abnehmbarer Deckel, eine Debug-Schnittstelle, oder ein Klartextschlüssel, der in Ruhe im Flash liegt.

Die Grenze, die bleibt: Durchsatz

> **Entscheidung vorweg, damit die Tabelle unten nicht missverstanden wird: Der Rechenkern wird NICHT gewechselt. Der nRF52840 bleibt.** Die Zahlen stehen hier, damit später jemand weiß, was ein Wechsel brächte — nicht, weil einer geplant wäre.

Der Rechenkern schafft am USB **rund 400 kB/s** — Bulk-Endpunkte sind auf 64 Byte je Mikroframe begrenzt (Nordic). Die microSD läuft über SPI mit einer Datenleitung: **rund 3 MB/s**.

	heute, und dabei bleibt es	*hypothetisch, bei nRF54LM20*
1 GB	42 min	*5,6 min*
16 GB	11,1 h	*1,5 h*

Warum nicht gewechselt wird: Ein Wechsel brächte Faktor 7,5 — danach bremst der Speicher, und **kein Chip mit BLE *und* NFC hat einen 4-Bit-Kartenanschluss**. Das in Frage kommende Modul (BL54LM20B) ist Vorserie: Beta-Muster, kein Preis, Zulassungen teils offen. Und der Wechsel hieße neue Platine, neue Längenrechnung, neue Brettdefinition.

Wann die Frage wieder aufgemacht wird: sobald das Modul lieferbar ist. Dann sprechen 512 KB RAM statt 256 und 2 MB Flash statt 1 dafür — und eine Zahl, die heute fehlt, entscheidet mit: **wie schnell AES-256-GCM am Gerät läuft.** Liegt sie unter 3 MB/s, wäre auch nach einem Wechsel nichts gewonnen.

> **Folge fürs Handbuch:** Die **29,10 MB** portabler Programme brauchen beim Lesen rund **73 Sekunden**. „Einstecken, starten" ist einstecken, über eine Minute warten, starten. **Die Zahl steht in** `docs/laufwerk.md`, alles andere verweist dorthin. Hier standen bis zum 30.10.2026 „27 MB" und „68 Sekunden" — die **gerechneten** Werte vom 23.10., während die **gemessenen** 29,10 MB längst daneben lagen. Abschnitt 6 dieses Plans führte den Widerspruch ausdrücklich als Punkt 6 auf, und dieser Absatz benutzte trotzdem die alte Zahl weiter. **Ein Widerspruch, den man aufgeschrieben hat, ist nicht behoben** — das ist der Grund für die Regel „eine Zahl nur an einer Stelle", und sie hat den Autor dieses Plans erneut erwischt. *(Die 400 kB/s, mit denen gerechnet wird, sind an einem nRF5340-DK gemessen, nicht am nRF52840 dieses Geräts — `docs/rechenkern-durchsatz.md` sagt das ausdrücklich und nennt es die einzige echte Messung.)*

---

3. Was das Gerät können muss

Anmelden und Ausweisen

FIDO2/WebAuthn/Passkeys · U2F/CTAP1 · PIV · OpenPGP · TOTP · HOTP · Yubico-OTP · Challenge-Response HMAC-SHA1 · statisches Passwort · hmac-secret/PRF · largeBlob · credProtect · minPinLength · Enterprise Attestation · mTLS · EAP-TLS · Kerberos/PKINIT · SSH mit Zertifikaten · PAM

Wege zu fremder Software

Einen Standard, über den jede App Passwörter aus einem Stick holt, gibt es nicht. Darum alle Wege, die es gibt:

Weg	erreicht damit
Gerät tippt als USB-Tastatur	jede Software, auch Anmeldebildschirm und BIOS
PKCS#11	Firefox, Thunderbird, OpenSSL, OpenSSH, Java, VPN
Minidriver + CNG (Windows)	jede Windows-App, Windows-Anmeldung
CryptoTokenKit (Mac/iPhone)	Safari, Mail, Keychain
ssh-agent	`ssh`, `git`, Ansible
FIDO-SSH (`sk-ecdsa`)	OpenSSH ohne Zusatzprogramm
Browser-Erweiterung	Chrome, Firefox, Edge
Kommandozeile `osk get`	Skripte, CI, Docker
lokale Schnittstelle	eigene Programme

> **Und wenn eine Software nur Dateien kennt — PEM, PFX, `.key`?** Das ist der häufigste Fall, an dem Hardware-Schlüssel scheitern, und die Antwort hat zwei Hälften: **Das Zertifikat kann das Gerät herausgeben**, als PEM oder DER. Ein Zertifikat ist öffentlich — es weiterzugeben kostet nichts. Für alles, was nur die Bescheinigung braucht (Serverkonfiguration, Vertrauensspeicher, eine Kette zum Mitliefern), genügt das. **Den privaten Schlüssel gibt es nicht als Datei. Nie.** Software, die zwingend eine Schlüsseldatei verlangt — ein `.pem` mit `PRIVATE KEY` darin, ein `.pfx` mit Passwort — **kann dieses Gerät grundsätzlich nicht bedienen.** Das ist kein Mangel, sondern sein Zweck: Was als Datei vorliegt, ist kopierbar. **Der Ausweg ist in fast allen Fällen PKCS#11**: OpenSSL, nginx, Java, VPN-Clients und Signierwerkzeuge nehmen statt eines Dateipfads eine Schlüssel-Adresse (`pkcs11:object=...`) entgegen. Wer das kann, braucht keine Datei. Wer es nicht kann, bleibt außen vor — **und das gehört ins Handbuch**, sonst sucht jemand stundenlang nach einer Exportfunktion, die es absichtlich nicht gibt.

Tresor

Eintragstypen: Login · TOTP · HOTP · SSH-Schlüssel · Zertifikat · PGP-Schlüssel · API-Schlüssel · Wiederherstellungscodes · Wallet-Seed · Lizenz · Karte · Identität · WLAN · Notiz · Datei

4 Profile mit je eigener PIN · 768 Einträge je Profil · AES-256-GCM · Suchmarke ohne Entschlüsseln · stromausfallsicher · Formatwanderung bei Updates · Geräteprotokoll (512 Einträge) · verschlüsselte

Sicherungsdatei mit **Shamir-Umschlägen**

Alltag: Passwortgenerator · doppelte, kurze und alte Passwörter finden · Ordner, Etiketten, Suche · Phishing-Schutz (die Erweiterung füllt nur auf der hinterlegten Adresse) · Oberfläche in fünf Sprachen

Geldbörse

Nur Bitcoin und Ethereum. (Frühere Zusagen zu Solana und Cardano sind nirgends umgesetzt und gestrichen.) BIP39/BIP32/BIP44 · PSBT · EIP-155/1559/712 · Ledger-verträgliche APDU-Schicht für MetaMask und Electrum · **Blindsignieren wird verweigert** — was das Gerät nicht lesbar anzeigen kann, signiert es nicht. Taproot wird ausdrücklich abgelehnt: „die Hälfte davon wäre gefährlicher als nichts."

Dongle und Server (NGC)

Challenge-Response gegen das Gerätezertifikat · **Entschlüsselung geschützter Programmteile** (nur die zweite Stufe ist echter Schutz; eine reine Ja/Nein-Abfrage lässt sich herausschneiden) · Bibliothek `osk-guard` für C, Go, PHP, .NET, Python · Knotenschlüssel im Chip statt auf der Platte · Dauerbetriebsmodus für Server, ausdrücklich einzuschalten und protokolliert.

Datenträger — drei Betriebsarten

Die microSD wird **nur in unseren eigenen Sticks** gelesen, nie in einem fremden Kartenleser. Das Format auf der Karte ist deshalb frei wählbar; der Stick zeigt dem Rechner ein gewöhnliches Dateisystem und übersetzt.

	Wer kann lesen	Geheim?
1 — nur dieser Stick	nur der Stick, der beschrieben hat	ja
2 — mit Kartenwort	jeder openSecureKey, der das Wort kennt	ja
3 — nur gesiegelt	jeder openSecureKey	**nein**

Alle drei haben **Merkle-Baum und Signatur**; der Unterschied liegt nur darin, wie — und ob — der Datenschlüssel eingewickelt ist. Empfängermodell wie `age`: Chipschlüssel, Kartenwort, oder der öffentliche Schlüssel eines anderen Sticks.

> **Die Anzeige muss den Modus nennen, bevor geschrieben wird.** Wer glaubt verschlüsselt zu haben und hat nur gesiegelt, gibt Geheimnisse weg — der schlimmste Fehler, den dieses Gerät machen kann, und er entsteht aus einer Verwechslung, nicht aus einem Angriff.

Zwei Grenzen, die ins Handbuch gehören:

1. **Echtheit ist nicht Urheberschaft.** Die Signatur beweist, dass die Daten unverändert sind. Dass sie von einer bestimmten Person kommen, beweist sie nur, wenn der Empfänger deren öffentlichen Schlüssel vorher kennt.
2. **Geprüft wird beim Lesen, nicht beim Einstecken.** Eine 16-GB-Karte vollständig zu prüfen hieße, sie ganz zu lesen — **1,5 Stunden**. Der Merkle-Baum prüft je Block nur dessen Pfad. Ein veränderter Block in einer Datei, die niemand öffnet, bleibt unentdeckt. „Karte geprüft" wäre eine Lüge, „dieser Block geprüft" die Wahrheit.

Verschlüsseln, signieren, Zertifikate

- **Dateien:** `age` **und** OpenPGP-Nachrichten, mit Blockverfahren. `age` passt gut — X25519 und ChaCha20-Poly1305 haben wir, es hat eine Erweiterungsschnittstelle für Hardware-Schlüssel und seit 2025 post-quanten-hybride Empfänger.
- **Ordner und Linux-Laufwerke fallen uns zu:** `gocryptfs` (`-fido2`) und LUKS2 (`systemd-cryptenroll`) brauchen nur **hmac-secret**, und das ist fertig. Kein eigenes Container-Format bauen. BitLocker nutzt denselben Mechanismus — belegt ist das für Windows aber nicht, also messen statt behaupten.
- **Zertifikate: CSR nach PKCS#10 auf dem Gerät** — die Grundfunktion, ohne die kein Zertifikat zu einem Chip-Schlüssel entsteht. Dazu Kette, X.509-Erweiterungen (SAN, Key Usage, EKU), **Widerrufsweg (CRL oder OCSP)** und Ablaufwarnung. **Eine CA ohne Widerruf wird nicht gebaut.**
- **Signaturen:** `sshsig`, Git-Commit-Signatur, **RFC-3161-Zeitstempel**.

Vorsorge

- **Stick zu Stick** über USB — der Hauptweg bei Verlust. Beide zeigen dieselbe Zahl, beide bestätigen mit Tastendruck.
- **CXF** (FIDO Credential Exchange Format, Standard seit 8/2025) für Import *und* Export von Passkeys, Passwörtern, TOTP und SSH-Schlüsseln. Schlägt zwei Lücken auf einmal: der Weg hinein aus fremden Managern und der Weg hinaus.
- **Verschlüsselte Sicherungsdatei** mit Shamir-Umschlägen für Familie oder Notar.
- **Importwege** aus KeePass, CSV und Browsern; Leak-Prüfung über k-Anonymität.
- Dienste-Liste je Stick, Sperrvermerk bei Verlust.

> **Spannung, die ins Bedrohungsmodell gehört:** Export heißt, dass Zugangsdaten das Gerät verlassen können. Das widerspricht „Schlüssel verlassen den Chip nie" — für Passkeys ist es mit dem Modus „gespiegelt" ohnehin entschieden. CXF macht es nur sichtbar und verlangt dieselbe Freigabe: PIN, Tastendruck, und eine Anzeige, die sagt **wohin**.

---

4. Sicherheit

Warum ein zweiter Chip: Die Auslesesperre des nRF52840 wurde schon durch Spannungs-Störimpulse überwunden. Darum liegt **kein Geheimnis dauerhaft im Rechenkern**.

- Schlüssel entstehen im Sicherheitschip und verlassen ihn nie
- Leitung dorthin verschlüsselt (SCP03) — **die Werksschlüssel sind öffentlich bekannt, der Austausch beim ersten Start ist Pflicht.** Genauer gemessen am 30.10.2026: `osk_se050_rotate_scp_keys()` **ist gebaut und geprüft**, aber **niemand ruft sie** — Fundstellen gibt es nur in der Deklaration, in zwei Attrappen, im Prüfsatz und im CHANGELOG. Das Werkzeug ist da, der Ablauf fehlt. „Fehlt" klang nach „nicht gebaut" und hätte den falschen Aufwand geschätzt; ein Gerät, das so ausgeliefert wird, läuft trotzdem mit öffentlich bekannten Schlüsseln
- Fehlversuchszähler im Chip, nicht im Flash
- Jede Herausgabe: PIN (je Sitzung) **und** Tastendruck (je Vorgang)
- **Automatisches Sperren** nach Zeit, **sofortiges Sperren beim Abziehen**
- Besonders geheime Einträge nur an **gekoppelten Rechnern**; dazu ein schreibgeschützter **Gastmodus**, in dem nichts geändert werden kann
- Das Programm zeigt immer, **welche Software gerade fragt**
- **Zurücksetzen auf Werkszustand** nur mit langem Tastendruck und getippter Bestätigung; **Selbstlöschung nach n Fehlversuchen**, abschaltbar
- Vergleiche mit gleicher Laufzeit, Zufallsverzögerung — **genau diese Fehlerklasse hat 2024 den Marktführer getroffen**: EUCLEAK, jeder YubiKey 5 vor Firmware 5.7 war klonbar (Infineon SLE78, nicht-zeitkonstante Inversion)
- RAM nach jedem Vorgang gelöscht, kein dynamischer Speicher in Krypto-Pfaden
- Parser für APDU, CTAP und NDEF werden gefuzzt
- Kein Server, keine Wolke, keine Telemetrie

Post-Quanten

NIST-Standards seit 8/2024. Kein Hardware-Schlüssel am Markt hat sie, und der Sicherheitschip kann sie nicht.

Entschieden: hybrid. ML-KEM **zusammen mit** X25519, gerechnet im Rechenkern (55 ms, ~11 kB Flash). Betroffen: Stick-zu-Stick, Server-zu-Server, CXP-Export. **Nicht betroffen:** die Sicherungsdatei — sie ist symmetrisch, AES-256 gilt als quantensicher.

> **Regel: hybrid oder gar nicht.** Fällt der PQC-Teil, bleibt X25519 aus dem Chip und nichts ist schlechter als heute. Ein reiner PQC-Pfad wäre ein Rückschritt, weil der Schlüssel dann nicht im Chip läge.

FIPS 140-3 Level 3

Arbeitsregel von Thomas, gilt dauerhaft und für alle Projekte: Wird ein Standard genannt, werden seine **technischen Anforderungen gebaut**. Nur der bezahlte Prüflauf wird geschoben — und es wird nichts getan, was ihn später verbaut.

Drei Punkte betreffen die Platine sofort:

1. **Epoxid-Verguss** über allen sicherheitsrelevanten Bauteilen — er ersetzt die Tamper-Schaltung. Platz, Höhe, Wärmeabfuhr und ein Fertigungsschritt gehören eingeplant; Prüfpunkte müssen danach unerreichbar sein. **Später nicht nachrüstbar.**
2. **Kein abnehmbarer Deckel.**
3. **Keine Debug-Schnittstelle** — allein ihre Existenz löst die Pflicht zur Reaktionsschaltung aus, und damit käme der Akku zurück.

Für die Firmware:

- **Die PIN darf nie über USB laufen.** Ab Stufe 3 ist ein vertrauenswürdiger Kanal Pflicht — damit wird die **PIN-Eingabe am Gerät** (Anzeige + Taster) von einer guten Idee zur Anforderung. Offen ist nur die Bedienung: eine sechsstellige PIN mit *einem* Taster durchzublättern ist zäh; zwei Taster oder langer/kurzer Druck sind zu prüfen.
- **Nie ein Klartextschlüssel in Ruhe:** PIN → langsame Ableitung → KEK → DEK.
- **Ed25519/X25519 als abschaltbare Nicht-FIPS-Erweiterung.**
- **Ein abgegrenzter, selten geänderter Sicherheitskern.** Zertifizierung und laufende Entwicklung vertragen sich nicht: Jede Firmware-Änderung macht einen Prüflauf ungültig. Das Vorbild erkauft seine Stufe damit, **gar keine Aktualisierung** zuzulassen. Für uns: signierte Firmware mit Prüfung im Gerät, Kern klein halten, Oberfläche außerhalb.

Level 4 ist als Geräte-Eigenschaft nicht baubar (Erkennungshülle braucht Dauerstrom; mehrfaktorige Authentifizierung ist mit einem Taster nicht erfüllbar). **Als Chip-Eigenschaft ist physische Sicherheit Stufe 4 schon erreicht** — aktives Schirmgitter, ohne eigene Energiequelle.

Zur Einordnung: Der **YubiKey 5 FIPS** erreicht Overall Level 2 mit Physical Security Level 3. Overall Level 3 hat bei Yubico nur das YubiHSM. **Unser Vorbild ist der datAshur PRO** — ein Stick, der Overall Level 3 mit Verguss statt mit Elektronik erreicht.

Ein Gegensatz bleibt und ist nicht auflösbar: Eine FIDO-Zertifizierung verträgt sich nicht mit gespiegelten Passkeys. Entweder ist das Zweitgerät ein echter Ersatz, oder die Bescheinigung ist gerätegebunden.

---

5. Was gebaut wird

Aus dem Marktabgleich gegen Yubico (alle Produkte inkl. YubiHSM 2), Nitrokey, SoloKey und OnlyKey: **achtzehn Punkte, sechs Stränge.**

Die Stränge — je ein Ordner, keine Überschneidung

	Ordner	Aufgaben
A	`se050/`	**RSA** (2048/3072/4096), **monotone Zähler** (AN13187), **SCP03/SCP11 zur Karte** samt Schlüsselaustausch beim ersten Start
B	`ctap/`	**CredentialManagement (0x0A)**, **Config (0x0D)**, **Enterprise Attestation**, **Signaturzähler in den Flash**, **PIN-Regelwerk**
C	`applets/`	**RSA in PIV und OpenPGP**, **statisches Passwort**, **PIV-Attestierung je Steckplatz (F9)**, **eindeutige Seriennummer**
D	`store/`	**Rückdrehschutz** über Chipzähler, **hash-verkettetes Protokoll**, **Abstreitbarkeit**, **Selbstzerstörungs-PIN**, **Kapazitäten**
E	`transport/`	**Yubico-HID-Report-Protokoll**

	Ordner	Aufgaben
F	neue Schicht	**16 Rechtebereiche**, **mehrere Zugangsschlüssel**, **YubiHSM Auth**, **RFC-3161-Zeitstempel**

Entschieden: alle sechs gleichzeitig, und die Arbeit läuft weiter, während die bestellten Teile unterwegs sind.

Die Vorarbeit, ohne die alles kollidiert

`include/osk_se050.h` **kennt weder RSA noch Zähler** (gemessen). Beides brauchen mehrere Stränge; dasselbe gilt für den Signaturzähler (gezählt wird in CTAP, gespeichert im Tresor).

Darum zuerst, vom Master allein: die Schnittstellen festlegen — zwei Header, keine Umsetzung. Danach bauen alle Stränge dagegen, auch wenn die Umsetzung dahinter fehlt.

> **Der Nebeneffekt ist mehr wert als die Vorarbeit kostet:** Alle Stränge arbeiten gegen `osk_se050.h` und sehen nicht, was dahinter antwortet. Wird der Sicherheitschip je getauscht, ändert sich eine Schicht — nicht fünf.

Querschnitt, kein eigener Strang

- **Zwei Firmware-Varianten** (mit und ohne RSA) als Kconfig-Option. Folgen: zwei Signaturen, zwei Prüfläufe, **zwei AAGUIDs** (FIDO verlangt, dass ein AAGUID eine Fassung mit denselben Fähigkeiten bezeichnet), und die Variante muss in Dateiname **und** Abbildkopf stehen, sonst spielt jemand die falsche auf.
- **Hybrides ML-KEM** für Stick-zu-Stick, Server-zu-Server und CXP.
- **Dateiwerkzeug** `osk encrypt` / `osk decrypt`. **Gebaut am 30.10.2026**, beide Formate (`age` und OpenPGP), ohne fremde Bibliothek. Geprüft: Rundlauf Byte für Byte, Verfälschung wird erkannt, falscher Empfänger scheitert. **Die Gegenseite im Chip ist eine Attrappe.** Die wichtigste ihrer Annahmen, und am Gerät zuerst zu prüfen: **ob der SE051 das rohe ECDH-Ergebnis herausgibt oder ein schon abgeleitetes.** Davon hängt ab, ob sich eine Datei später überhaupt öffnen lässt. Die übrigen stehen in `docs/dateiwerkzeug.md`.
- **CSR und CA-Verwaltung**, mit Widerrufsweg.
- **CXF-Import und -Export.**

Was nicht gebaut wird — mit Grund

	Warum nicht
Verschlüsselter Massenspeicher (Nitrokey Storage)	**erledigt, unter anderem Namen** — der microSD-Datenträger kann mehr: weitergeben und Echtheit prüfen
Fingerabdruck in Stufe 1	Bestückungsstufe 2. Bei FIDO2 *ersetzt* Biometrie die PIN — sie ist kein zusätzlicher Faktor. `BioEnrollment` bleibt abgelehnt: keine Website darf einen Finger einlernen
Lightning	proprietär, von USB-C abgelöst
FIDO über Bluetooth	Browser unterstützen es nicht verlässlich; BLE dient nur dem eigenen Programm
Eigenes Container-Format	gocryptfs und Cryptomator existieren und brauchen uns nur als Schlüssel
PAdES/CAdES/XAdES	später; die höheren Stufen hängen an der CA
M von N — den Schlüssel in Teile zerlegen	**entschieden am 27.10.2026: nein**, aus drei Gründen. Siehe Kasten
JavaCard als Sicherheitschip	**entschieden: nein.** JCOP-Chips gibt es nicht im Handel — NXP liefert nur Wafer oder Module an Kartenhersteller, über Direktvertrieb. Im Handel sind nur fertige Karten

> **Warum M von N nicht gebaut wird — der wichtigste Verzicht in diesem Plan:** *Die Idee:* Der Schlüssel wird in **N Teile** zerlegt (etwa sieben Umschläge, nicht Geräte), und **irgendwelche M** davon (etwa vier) öffnen ihn wieder. Damit übersteht ein Wurzelzertifikat den Verlust des Geräts, ohne dass ein einzelner Verwalter allein herankommt. **1. Es macht das Gerät angreifbarer, nicht sicherer.** Heute kann ein Schlüssel den Chip **gar nicht** verlassen — wer das Gerät stiehlt und aufbohrt, bekommt nichts. Damit Teile entstehen können, muss der Schlüssel erst heraus. Danach liegt er in sieben Umschlägen in der Welt statt in einem Chip. **2. Es löst ein Problem, das dieses Gerät nicht hat.** Der Verlustfall ist dreifach abgedeckt: Zweitstick über Stick-zu-Stick, verschlüsselte Sicherungsdatei mit Shamir-Umschlägen, und für ein Wurzelzertifikat der klassische Weg — offline erzeugt, auf Papier, im Tresor. Das ist sicherer als jedes Gerät. **3. Die Hardware dafür ist nicht beschaffbar.** Der SE051 kann keinen Schlüsselexport (AN12413), und JavaCard — der einzige Weg dorthin — ist als Chip nicht zu kaufen. **Was bleibt:** das vorhandene Shamir-Verfahren auf dem **Dateischlüssel** der Sicherung (`osk_shamir.c`), etwa 3 von 5. Ein echter Schwellenwert, aber auf einer Datei statt auf dem Chipschlüssel — und genau deshalb ungefährlich.

---

6. Die acht Widersprüche in den eigenen Unterlagen

Vor allem anderen zu klären. Solange die Unterlagen sich widersprechen, ist jede Planung auf Sand gebaut. Die vier schwerwiegenden:

1. **Ist für ARM übersetzt worden?** `docs/build.md` und `architecture.md` sagen ja, mit gemessenen Zahlen (161 124 B). `docs/update.md` und `CHANGELOG.md` sagen nein. **Davon hängt ab, auf welcher Stufe das Projekt steht.**
2. **Ist PIV, OpenPGP, Wallet, Anzeige, Funk geschrieben?** `architecture.md` sagt nein; die vier Einzeldokumente beschreiben alles als geschrieben und getestet, mit Dateilisten.
3. **Wie viele Credentials?** `ctap.md` sagt unbegrenzt (abgeleitet), `architecture.md` sagt endlich (Chipspeicher). Genau die Zahl, die Yubico mit 100 Passkeys beziffert.
4. **Nutzerspeicher: 46, 50 oder 104 kB?** Drei Dokumente, drei Zahlen. *Teilantwort aus AN12973: es ist eine Spanne, 46–104 je nach Konfiguration.*

Und die vier übrigen, damit die Zahl im Titel stimmt:

5. **War die Auslagerung des Update-Platzes nötig?** `architecture.md` und `update.md` begründen ausführlich, warum der zweite MCUboot-Platz ins externe Flash muss — und halten danach fest: „war nicht nötig".
6. **27,5 oder 29,10 MB** für die drei portablen Programme? `hardware.md` rechnet 27,5 (23.10.), `laufwerk.md` misst 29,10 (24.10.). Aus „34,5 MB frei für den Tresor" werden damit 28,2 plus 6,82 MB Reserve.
7. **PKCS#11-Freigaben:** Dieselbe Datei sagt an einer Stelle, ssh-agent und PKCS#11 holten noch jeden Schlüssel einzeln — und zwei Abschnitte höher, das sei am 24.10. geschlossen worden.
8. **Kann der gelieferte Chip Ed25519, X25519 und ECDH?** Die FIPS-Politik eines echten SE051 zeigt die Maske `0x27F2`: EdDSA aus, Curve25519 aus. Die Variantentabelle AN12973 sagt das Gegenteil. **Entschieden wird es am gelieferten Teil** — Kennung lesen, einmal, und die Frage ist weg.

> **Das Muster dahinter, in diesem Projekt neunmal aufgetreten: zwei Größen, die gleich heißen und Verschiedenes messen.** Padmitten gegen Kupfer. Kontaktzahl gegen Pinzahl. Gigabit gegen Gigabyte. Luftlinie gegen verlegte Länge. Gehäusetiefe gegen Lötbildtiefe. Zellenmaße mit und ohne Schutzplatine. Geräteweiter Signaturzähler gegen Zähler je Nachweis. Bauteilhöhe gegen Gehäusedicke.

Sechs der acht sind am 30.10.2026 aufgelöst, jede Zahl steht nur noch an einer Stelle:

	aufgelöst wie	gilt jetzt in
1	Der ARM-Bau ist am 25.10.2026 wirklich geschehen — `update.md` und `CHANGELOG` waren vom 24.10. und überholt. **Nachgemessen:** zwei Werkzeuge, und die Rechnung geht auf (160 148 + 976 = 161 124 B), Board `nrf52840dk/nrf52840`. Das Projekt bleibt auf **Stufe 2 von 4**	`docs/build.md`
2	PIV, OpenPGP, Wallet, Anzeige, Funk sind **geschrieben und übersetzt**; `architecture.md` sagte noch „nichts davon geschrieben". Offen bleibt nur die Zephyr-Anbindung und `osk_ui_port_nrf.c`	`docs/architecture.md`

	aufgelöst wie	gilt jetzt in
3	Zwei Größen gleichen Namens: **abgeleitete** Nachweise (gespiegelt, unbegrenzt) gegen **je ein Schlüsselpaar im Chip** (nur dieses Gerät, endlich)	`docs/architecture.md`
5	Die Auslagerung des Update-Platzes war **nicht nötig** und bleibt als Reserve	`docs/architecture.md`
6	Gültig ist die **Messung** (`laufwerk.md`, 29,10 MB), nicht die Rechnung; `hardware.md` ist als überholt gekennzeichnet	`docs/laufwerk.md`
7	Der offene Punkt war **erledigt**: der Code liest den öffentlichen Teil, `reveal` ist nur Rückfall	`docs/anbindungen.md`

Zwei bleiben offen, und das mit Absicht: 4 (Nutzerspeicher) und 8 (kann der Chip Ed25519, X25519, ECDH) entscheidet erst der gelieferte Chip. Dort zu raten wäre eine erfundene Entscheidung, und die ist schlimmer als ein offener Punkt.

---

7. Stufen

Jede Stufe endet mit einer **Messung am Gerät**, nicht mit einem grünen Bau.

	Inhalt	Prüfung
1	Teststand: SDK, Bau, Brett läuft	USB erkannt, Taster/LED
2	**Sicherheitschip lebt**	**Kennung lesen: A8FA/A564 = richtig, A920/A565 = falscher Chip.** Schlüssel erzeugen, signieren, am PC nachrechnen; **am I²C mitlesen — kein Klartext**
3	FIDO2 über USB, **CredentialManagement, Config, Enterprise Attestation, Signaturzähler im Flash, PIN-Regelwerk**	echter Passkey, Chrome und Windows Hello. Dazu: LUKS2 und gocryptfs entsperren
4	NFC	Anmeldung am Handy
5	Smartcard: PIV, OpenPGP, OTP, **RSA**, **Yubico-HID-Transport**, **statisches Passwort**, **PIV-Attestierung**, **CSR**, **Seriennummer**	Zertifikat, E-Mail signieren, SSH, **KeePassXC und VeraCrypt entsperren**
6	Tresor, **Rückdrehschutz**, **verkettetes Protokoll**, **Kapazitäten**	Passwort getippt, TOTP gegen zweite App, **zurückgespielter gültiger Zustand wird erkannt**
7	Verwaltungsprogramm	Windows, dann Linux, dann Mac
8	Anbindungen, **Dateiwerkzeug** (`age`, OpenPGP), **sshsig**, **Git-Signatur**	je eine echte Operation: `git push`, `openssl`, Firefox, Erweiterung
9	Bluetooth	Kopplung mit Zahlenvergleich
10	Eigene Platine, **Verguss**	DRC 0, Gerber nachgemessen
11	Anzeige, **PIN-Eingabe am Gerät**	richtige Webseite am Gerät — und bei untergeschobener Anfrage die falsche, sichtbar
12	Fingerabdruck	Anmeldung allein mit Finger
13	Geldbörse	Testnetz-Überweisung aus Electrum und MetaMask
14	Dongle und Server, **YubiHSM Auth**, **Rechtebereiche**	Programm startet nur mit Stick; NGC signiert einen Auftrag
15	Laufwerk	an fremdem Rechner ohne Adminrechte starten
16	iPhone, dann Android	
17	Alltag und Vorsorge, **CXF**, **Importwege**, **Abstreitbarkeit**, **Wisch-PIN**	Zweitstick, Sicherung, Import aus KeePass
18	Aktualisierung und Herstellung	MCUboot A/B, reproduzierbarer Bau
—	**Eigene CA** (Kette, Widerruf, Ablaufwarnung) · **Datenträger** (drei Betriebsarten) · **ML-KEM** als Querschnitt	eigene Stufen, Einordnung offen

Der signierte Bootloader kommt früh (mit Stufe 3 ins Abbild) — sonst müsste später die Auslesesperre gelöst werden, und das löscht den Chip.

---

8. Prüfung

Vier Stufen von „geprüft": geschrieben / übersetzt / im Abbild / am Gerät gelaufen. Im Protokoll steht, welche.

- Jeder Strang liefert PC-Tests; **keiner meldet mehr als Stufe 2**, bis Hardware da ist.
- Neue Parser werden gefuzzt — das Yubico-HID-Protokoll ist ein Parser, den ein Angreifer unmittelbar erreicht.

Drei Dinge, die sonst durchrutschen:

1. **Der Signaturzähler muss einen Neustart überstehen.** Heute steht er im RAM und fängt bei null an; ein Test, der nur innerhalb eines Laufs zählt, zeigt den Fehler nicht.
2. **Der Rückdrehschutz muss gegen einen zurückgespielten, in sich gültigen Zustand prüfen**, nicht gegen einen verfälschten. Genau das ist der Angriff.
3. **Beide Firmware-Varianten** müssen gebaut und geprüft werden. Eine Option, die nur in einer Richtung getestet ist, ist keine.
4. **Eine Attrappe prüft die eigene Seite des Vertrags, nicht die andere.** Am 29.10.2026 am Signaturzähler aufgetreten: CTAP spricht den geräteweiten Zähler mit der reservierten Kennung 0 an, der Tresor sucht zu jeder Kennung einen Indexsatz und findet für die 0 keinen — am Gerät wäre **gar nicht signiert** worden. Grün war trotzdem alles: die Attrappe in `test/ctap/mocks.c` führt selbst einen geräteweiten Zähler und übergeht die Kennung. **Zwei Größen, die gleich heißen und Verschiedenes messen — das achte Mal in diesem Projekt**, und das erste Mal zwischen zwei Strängen statt in einer Unterlage. **Entschieden am 29.10.2026:** Kennung 0 ist der geräteweite Zähler und liegt als `sigcount_dev` im Kopfsatz (vier Byte aus `reserved[16]`, crc32 bleibt letztes Feld); jede andere Kennung zählt weiter je Eintrag. Das deckt alle Nachweise ab — nicht auffindbare Nachweise und U2F haben keinen Indexsatz, in dem ein Zähler liegen könnte, und das sind die meisten. Offen daraus: **jede Anmeldung kostet einen Kopfsatz** im Flash, womöglich zusätzlich einen NVM-Schreibvorgang im Sicherheitschip. Beides ist gemeldet, nicht gerechnet.

Was in 8 MB läuft, beweist nichts für 64 MB — Adressbreite, Löschblockgröße und Verschleißverteilung ändern sich mit der Größe. (Das Entwicklungsbrett hat 8 MB, der Entwurf 64.)

Werkzeugergebnisse sind erst Belege, wenn geprüft ist, was sie messen: `pcb_layout_boxes` misst Pad-Mitten statt Kupfer, die DRC nennt je Gießflächen-Insel nur einen Vertreter, und eine Bändertabelle aus Padlagen sieht **kein Bauteil ohne Pads** — kein Anzeigenglas, keine Folie, keinen Sensor, der nur aufliegt.

---

9. Offene Punkte

Vor der Bestellung der Platinenteile:

1. **Händler-Bestellnummer des SE051C2.** Die Variante ist belegt (AN12973 Tab. 3), die Nummer für die Serie nicht. Eine Typnummer aus der Wildnis: `SE051C2HQ1/Z01XD` (`ungeprueft`, aus einer Boardbeschreibung).
2. **Flash-Rechnung.** Passen RSA, ML-KEM, age, OpenPGP, CXF, CSR und die CA-Verwaltung neben MCUboot in 1 MB? Hochrechnung heute: 264 kB von 960.
3. **Wie viele RSA-4096-Schlüssel** in den Nutzspeicher passen — hängt an Widerspruch 4.
4. **AES-256-GCM-Durchsatz am Gerät.** Liegt er unter 3 MB/s, ist er die neue Wand für den Datenträger. Nur am Gerät messbar.

4a. **Stapelbedarf des SE050-Treibers nach der erweiterten APDU-Länge.** Gefunden beim Zusammenführen von Strang A am 28.10.2026. Damit ein RSA-4096-Chiffrat (512 Byte) durchpasst, sind drei Puffer auf `SE050_MAX_PAYLOAD` = 889 Byte gewachsen: `cmd[898]` in `plain_exchange()` sowie `payload[905]` und `plain[896]` in der SCP03-Schicht. **Zusammen knapp 1,8 kB auf dem Stapel eines Fadens**, der unter Zephyr oft 1–2 kB groß ist. **Das ist kein Übersetzungsfehler und fällt am PC nicht auf** — dort ist der Stapel groß. Am Gerät ist es entweder ein Absturz oder eine Stapelwächter-Meldung. Zwei Wege: Fadenstapel vergrößern, oder die Puffer aus

dem Stapel in einen festen Bereich verlegen. **Entschieden wird es an einer Messung** (`CONFIG_THREAD_ANALYZER`), nicht an einer Schätzung — und das geht erst mit dem Teststand.

Ungelöst, zwei davon dauerhaft:

5. **Gespiegelte Passkeys gegen FIDO-Zertifizierung** — hier schließen sich zwei Anforderungen aus.
6. **Fünf USB-Schnittstellen gleichzeitig** — nie an echten Rechnern geprüft, manche BIOSe stolpern darüber. Gegenmittel: einzeln abschaltbar.
7. **Untergeschobene Firmware** — dagegen hilft nur der signierte Bootloader.
8. **Mehrere Profile sind in FIDO nicht vorgesehen** — beim Profilwechsel muss man sich bei den Diensten neu anmelden.
9. **Hauptkennwort vergessen und beide Sticks verloren = alles verloren.** Deshalb ist der Notfallzettel im Einrichtungsassistenten nicht überspringbar.
10. **Lizenz** (Software und Hardware getrennt), **USB-Kennung** über pid.codes, **AAGUID** eintragen.
11. **Tasterzahl für die PIN-Eingabe am Gerät** — einer ist zäh, zwei brauchen Platz.

Wo die Einzelheiten stehen — dieser Plan wiederholt sie nicht, sonst entstehen wieder zwei Wahrheiten:

Thema	Datei im Repo
Was zum Anfangen zu bestellen ist, nach Dringlichkeit	`docs/bestellliste-teststand.md`
Datenträger: Betriebsarten, Merkle-Baum, Kartenwort	`docs/datentraeger-konzept.md`
Durchsatz, Engstellen, Chipalternativen	`docs/rechenkern-durchsatz.md`
Stromaufnahmen mit Datenblattstellen	`docs/stromaufnahme.md`
FIPS 140-3: elf Bereiche als Bauliste, 21 Bauaufgaben	`D:\claude\side-session-notes\fips-140-3.md`
Maße, Bänder, Platzierung	`hardware/layout-notes.md`
Messmittel für die Platine (Kupfer am Punkt, Pad gegen Pad, Hülle des Funkmoduls)	`hardware/tools/LIESMICH.md`
Gehäusemaße, Lagensummen, warum 2,90 mm nicht die Gehäusedicke ist	`cad/masse.md`
Dateiwerkzeug `osk encrypt` / `osk decrypt`	`docs/dateiwerkzeug.md`
Wo sich SE050-Applet 3.x und 7.2 unterscheiden	`docs/se051-unterschiede.md`

Zum Aufräumen im Repo:

Erledigt am 27.10.2026: `docs/energiebudget.md` gelöscht (der einzige Chipwert daraus, 3,00 µA Deep Power-down, steht in `docs/stromaufnahme.md`) · `hardware/layout-notes.md` und `docs/fingerprint.md` berichtigt · Gegenprobe über das ganze Repo gelaufen.

Der Befund der Gegenprobe war größer als erwartet — und am 30.10.2026 hat sich gezeigt, dass er auch falsch gelesen war. Hier stand, die Höhengrenze sei „von 3,8 auf 2,90 mm gefallen" und der Sensor der Stufe 2 passe nur noch mit 0,1 mm Luft darunter. **Beides stimmt nicht**, weil die 2,90 mm etwas anderes messen als eine erlaubte Gehäusedicke:

- **2,90 mm ist die Höhe der Steckerhülle von J1 über alles**, symmetrisch zur Platinenmittelebene — das, was in eine USB-Buchse passen muss.
- **Die Gehäusedicke ist 7,03 mm** (`cad/masse.md`, Fall `stufe1-b`), und für sie ist überhaupt keine Grenze gesetzt. Eine von 2,90 mm wäre auch nicht baubar: U1 allein ist 2,05 mm hoch und steht auf der 0,8 mm dicken Platine, für Dach und Boden blieben 0,05 mm.
- **Die einzelnen Bauteile bleiben unter 2,90 mm** — das höchste ist U1 mit 2,05 mm. Diese Prüfung ist eingehalten.

Das ist der neunte Fall desselben Fehlermusters: zwei Größen, die gleich heißen und Verschiedenes messen — hier Bauteilhöhe gegen Gehäusedicke. Der Satz oben hat sie drei Tage lang verwechselt, und aus ihm wäre die falsche Entscheidung über den Fingerabdrucksensor gefolgt.

Was daraus offen bleibt:

12. ~~**Das Gehäuse muss neu gezeichnet werden.**~~ **ERLEDIGT am 30.10.2026.** `cad/masse.md` und `cad/gehaeuse.md` sind neu geschrieben, das Geometrieskript rechnet sechs Fälle durch. **Gehäusedicke 7,03 mm** (Fall `stufe1-b`, der gebaute). **Nachgemessen:** `cad/tools/budget.py` bestand 9 von 9 Kontrollpunkten, 4 davon mussten anschlagen. Dabei kam der Lesefehler zu den 2,90 mm heraus, der oben in Abschnitt 2 steht. **Gezeichnet und gerechnet — nichts gedruckt, nichts gemessen.**
13. **Bauhöhe des FPC2534 mit Fassung** — 0,83 mm ist der Sensorkörper allein. Die Frage hing bisher an der falschen Grenze: Mit der Gehäusedicke als Maß ist der Sensor **kein Engpass mehr** — der Fall `stufe2-b` kommt auf dieselben 7,03 mm wie der gebaute. Nur das deutlich dickere BM-Lite-Modul kostet 1,12 mm. **Trotzdem nicht ableitbar, sondern am Muster zu messen** — aber es entscheidet nicht mehr über das Gehäuse.
14. ~~`status.battery` **ist ein toter Zustand.**~~ **ERLEDIGT am 30.10.2026.** Aus Firmware, Sprachtabelle und fünf Sprachdateien heraus; vier Zustände bleiben (gesperrt, offen, arbeitet, Fehler). **Nachgemessen:** 30 Schlüssel in allen fünf Sprachen deckungsgleich **auch in der Reihenfolge**, der Erzeuger `generate.py --check` stimmt mit den erzeugten Dateien überein, UI-Prüfsatz **4074 Prüfungen, 0 Fehler**. Beim Nachmessen fand ich noch einen Kommentar in `osk_views.h`, der den Akku weiterführte — der Code war richtig, der Kommentar nicht.
18. **Der Akku ist im Repo-Stand NICHT entfernt.** Nebenbefund von Strang K, von mir nachgemessen: der Schaltplan-Export trägt **16 ×** `VBAT`**, 3 ×** `CHG`**, 3 ×** `STAT`, die Netzliste **19 ×** `VBAT` und 5 × `CHG`. Abschnitt 2 dieses Plans sagt seit dem 25.10.2026 „14 Bauteile sind aus Schaltplan und Platine entfernt", und `hardware/layout-notes.md` nennt die Zellen „entfällt seit 25.10.2026". **Beweisbar ist nur: der Stand im Repo trägt ihn** — die Exporte sind vom 24./25.10., und was heute in EasyEDA liegt, konnte ich nicht sehen (die Brücke antwortete die ganze Runde NO_EDA). Zu prüfen ist also zuerst, ob der Eingriff nie geschehen oder nur nie exportiert worden ist. **Solange das offen ist, darf keine Stückliste und kein Gerber als Fertigungsstand gelten.**

> **Das Muster hinter Punkt 18, und es ist ein anderes als die zwei gleich benannten Größen: eine Entscheidung wird als ausgeführt weitergereicht.** Dreimal in zwei Tagen gefunden — `status.battery` war entschieden und stand noch im Code; der SCP03-Schlüsseltausch war gebaut und wurde nie gerufen; der Akku ist entschieden und steht noch im Schaltplan. **Jedes Mal war die Unterlage richtig und die Wirklichkeit nicht.** Wer eine Entscheidung einträgt, hat sie nicht ausgeführt, und wer sie ausführt, hat sie nicht angeschlossen. Die Gegenprobe gehört an die Sache, nicht an das Papier.

19. **Fünf vollständige private Schlüssel stehen im öffentlichen Repo**, als Konstanten in Quellcode: vier in `apps/desktop/classes/agent_golden_test.go`, einer in `apps/desktop/tests/freigaben_test.go`, zusammen 58 Zeilen Schlüsselmaterial. Es sind Testschlüssel, und sie stehen dort aus einem nachvollziehbaren Grund — ein Golden Test braucht feste Werte, keine neu erzeugten. **Die Projektregel sagt trotzdem: Signaturschlüssel gehören nie ins Repo. Der eigentliche Befund ist, dass der eigene Schutz sie nicht sieht:** `tools/keyguard.py` meldet „Kein privater Schlüssel im Projektordner", und das ist aus seiner Sicht richtig — es prüft, ob **eine Datei** ein Schlüssel ist (Muster am Dateianfang), nicht, ob ein Schlüssel **in** einer Datei steckt. Sein eigener Selbsttest stuft `const key = "-----BEGIN PRIVATE KEY-----"` ausdrücklich als *keinen* Schlüssel ein, um Fehlalarme zu vermeiden. **Beides zusammen ist die Lücke: der Fall, den der Schutz bewusst auslässt, ist genau der Fall, der eingetreten ist. Zu entscheiden, nicht zu rechnen:** Schlüssel zur Laufzeit erzeugen (dann ist es kein Golden Test mehr), nur die öffentlichen Teile fest hinterlegen, oder es so lassen und ausdrücklich begründen. Den Schutz strenger zu machen erzeugt Fehlalarme in `keyguard.py` selbst.
20. **Ein schlüsselartiger Wert in Testdaten hat GitHubs Scanner ausgelöst** (Alert 1, „publicly leaked"): `apps/desktop/tests/vektoren/cxf-fido-beispiel.json` trug in einem erfundenen CXF-Beispieldatensatz einen Wert im exakten Format eines Google-API-Schlüssels. Am 30.10.2026 ersetzt durch einen, der kein Format nachahmt; kein Test hing daran. **Zwei Dinge bleiben offen.** Erstens: ob der Wert echt war, ist nicht geklärt — GitHub sagt `validity: unknown`. War er aus einem fremden Beispiel übernommen, hat dieses Repo einen fremden Schlüssel veröffentlicht. Zweitens: **er steht weiter in der Geschichte.** Ein öffentliches Repo vergisst nichts, solange die Geschichte nicht umgeschrieben wird, und das ist eine Entscheidung mit Folgen für jeden Klon. **Bemerkenswert:** Push Protection war am Repo **aktiviert** und hat den Push nicht verhindert (`push_protection_bypassed: false`). Ein Schutz, auf den man sich verlässt, ohne ihn zum Anschlagen gebracht zu haben, ist keiner — dieselbe Regel, die für die eigenen Prüfungen gilt. **Regel daraus:** Testdaten ahmen kein Schlüsselformat nach. Ist der Wert echt, ist er geleakt; ist er erfunden, macht er die Sicherheitsmeldungen des Repos unbrauchbar, weil er jedes Mal wieder anschlägt.

Daraus neu offen, beides beim Nachmessen am 30.10.2026 gefunden:

15. ~~`hardware/layout-notes.md` **trägt den Akku noch im laufenden Text.~~ ERLEDIGT am 30.10.2026 — und der Befund lag anders, als er hier stand.** Die Datei ist **ein Arbeitstagebuch von 3728 Zeilen in 90 Abschnitten**, chronologisch, das sich fortlaufend selbst berichtigt. Die Akku-Stellen sind deshalb nicht falsch, sondern **Geschichte**: man liest ihnen ab, warum eine Zahl so wurde. Sie zu löschen wäre der Fehler gewesen. **Was wirklich fehlte, war ein geltender Stand** — man konnte 3728 Zeilen nicht ansehen, um eine Zahl von heute zu erfahren, obwohl dieser Plan und `cad/parameters.csv` die Datei als Quelle führen. Sie hat jetzt einen Kopf: was gilt, was nicht gilt, und was trotz des Akkus weiter gilt. Zwei Stellen im laufenden Text, die sich nicht selbst als Geschichte zu erkennen gaben, sind gekennzeichnet.
16. ~~**Die Grenze „Stecker mittig ± 0,50 mm" ist nirgends belegt.~~ ZURÜCKGENOMMEN am 30.10.2026, der Vorwurf war zu scharf.** Sie ist begründet, im Kopf von `pruefe_mittig`: gesetzt, nicht gemessen, am Muster zu prüfen. Sie stand nur nicht dort, wo man nach Maßen sucht. Jetzt steht sie als `usb_center_tol` in `cad/parameters.csv` und das Skript liest sie von dort — vorher war sie ein Vorgabewert im Code, obwohl der Dateikopf sagt, alle Zahlen kämen aus der Parameterdatei.

Und der Punkt, der beim Aufräumen herauskam — der schwerste von heute:

17. **Die Platine ist 70,0 mm lang, nicht 59,55.** Gemessen am `BOARD_OUTLINE` in `hardware/pcb-source-PCB1.txt` (2755,9055 mil = 70,0 mm; die Breite bestätigt sich mit 27,0 mm). Die 59,55 mm sind die **gerechnete** obere Schranke, nachdem der Akku entfiel — der Abschnitt, der sie ausgerechnet hat, sagt ausdrücklich „die Platine ist nicht angefaßt, der Umriss unverändert". **Weitergereicht wurde trotzdem nur die Zahl**, in diesen Plan (Abschnitt 2) und in `cad/parameters.csv`. **Gegenprobe, und sie ist am 30.10.2026 von Strang K berichtigt worden:** Hinter `y` = 59,55 mm liegen **vier Bauteile mit zwölf Pads** — D1 (die RGB-LED, 6 Pads) und R10/R11/R12 (je 2). **Hier standen vorher „vier Pads", und das war mein eigener Messfehler:** ich hatte Bauteil-MITTEN gezählt, nicht Pads. Dieselbe Verwechslung noch einmal bei der hintersten Kante — die 66,0 mm sind die **Mitte** von D1, die **Kupferkante** liegt bei **66,985 mm**. Am Gerber gemessen, nicht am Platinen-Export; der Export nennt die Pads gar nicht, und mein Werkzeug hat das nicht gemerkt. *(Padmitten gegen Kupfer — der erste der acht Fälle in Abschnitt 6, und ich bin ihm selbst hineingelaufen.)* Die kürzere Platine ist also nicht bloß ungezeichnet, sondern mit der heutigen Platzierung unvereinbar. **Drei Wege, von Strang K gerechnet** (`hardware/layout-notes.md`): **A** auf 59,55 mm — 16 Pads neu, vier Gießflächen neu, DRC; kostet die NFC-Zone **42 % Fläche**, reißt aber keine schon gemessene Auflage. **B** auf 70,0 mm — eine Zahl in `cad/parameters.csv`, +10,45 mm Gehäuse, +1,23 ml Verguss. **C1** auf **67,30 mm** — die kürzeste Platine, bei der **kein Bauteil wandert**. **Folge: das neu gezeichnete Gehäuse ist 10,45 mm zu kurz** für die Platine, die im Repo liegt. Die Höhenrechnung bleibt gültig — Lagensummen laufen in `z`, dort ändert die Länge nichts. **Zu entscheiden, nicht zu rechnen:** entweder wandern die vier Pads und die Platine wird gekürzt, oder das Gehäuse wird länger. Beides ist Arbeit an der Platine. **Das ist der zehnte Fall des Fehlermusters: Platinenlänge gezeichnet gegen Platinenlänge gerechnet.**

---

10. Arbeitsweise

Repo: `Z:\GitHub\opensecurekey`, öffentlich, MIT, nach `claude-md\repo.md`. README englisch, Autor „Sorglos Thomas Weirich", vierstellige Version je Teilprogramm, Oberflächen in fünf Sprachen.

```
apps/firmware/   Zephyr · src/classes/{se050,ctap,applets,transport,store,ui,wallet}
apps/desktop/    Go, lokale Weboberfläche, `osk`
apps/web/        Browser-Erweiterung
shared/          Protokoll, Tresor-Datenformat, osk-guard
hardware/        Schaltplan, Layout, Gerber, Stückliste
cad/             Gehäuse (Fusion 360)
docs/            Architektur, Bedrohungsmodell, Messprotokolle, Handbuch
tools/           build.py, set_version.py, flash.py, release.py, md2pdf.py
```

Diese Sitzung bleibt Master: verteilt, sammelt, führt zusammen, fragt als Einzige. Jeder Strang bekommt einen eigenen Zweig und fasst nur seine Ordner an.

Regeln, die aus Fehlern dieses Projekts stammen:

- **Eine Zahl nur an einer Stelle.** Maße stehen in `hardware/`, alles andere verweist. Kopien veralten still — hier fünfmal passiert, einmal quer durch das Handbuch in fünf Sprachen.
- **Befund und Begründung trennen.** Fällt die Begründung, fällt der Befund nicht mit.
- **Nicht nach der erwarteten Zahl suchen** — das findet nur den vermuteten Fehler. Alle Maßzahlen auflisten und einzeln ansehen.
- **Geteilter Checkout:** HEAD vor jedem Commit prüfen, `git add` und `git commit --` zusammen, Zweig beim Push ausdrücklich nennen.
- **Ein Werkzeugergebnis ist erst ein Beleg, wenn geprüft ist, was es misst.**
- **Gemessen wird dort, wo das Repo liegt, und mit der Schale, die der Nutzer startet.** Am 29.10.2026 zweimal am selben Prüflauf aufgetreten: er lief im Worktree auf `D:`, aber nicht auf `Z:` (die WSL kann Netzlaufwerke nicht übersetzen) — und nach der Nachbesserung lief er in PowerShell 7, aber nicht in 5.1, weil dort die Fehlerausgabe eines fremden Programms den Lauf abbricht. **5.1 ist, was beim Doppelklick startet.** Beide Male war die gemeldete Zahl richtig und der Weg dorthin nicht nachvollziehbar.
- **Was nicht am Gerät lief, wird nicht als geprüft gemeldet.**

Dieser Plan liegt zweimal — unter `.claude\plans\` (Arbeitsfassung) und als `docs/plan.md` im Repo. **Jede Änderung endet mit zwei Schritten, die zusammengehören:** Kopie ins Repo ziehen und das PDF neu erzeugen (`tools/md2pdf.py`). Sonst entstehen zwei Wahrheiten — genau das ist am 27.10. passiert und hat den Autor dieses Plans als Ersten erwischt.